Equazioni di ricorrenza

Indice

Introduzione

Equazioni di ricorrenza

Il metodo di espansione

Il metodo di sostituzione

Il metodo dell'albero di ricorsione

Verso il Master theorem

Riepilogo: quale metodo quando

Introduzione

Introduzione: dalla ricorsione alle ricorrenze

- Nel capitolo *Ricorsione* abbiamo contato i passi di un algoritmo ricorsivo riga per riga
- Il numero di passi eseguiti da una chiamata dipende dal numero di passi eseguiti dalle sue chiamate ricorsive
- Queste sono chiamate dello *stesso* algoritmo su un input *più piccolo*
- La quantità che vogliamo calcolare è quindi definita in termini di se stessa: questa è una **equazione di ricorrenza**

Il piano del capitolo

- Definiamo prima le equazioni di ricorrenza e mostriamo da dove nascono
- Poi studiamo tre metodi per risolverle, cioè per ottenerne la forma chiusa o almeno l'ordine di grandezza:
  - il **metodo di espansione**, che srotola l'equazione fino a raggiungere la condizione al contorno;
  - il **metodo di sostituzione**, che dimostra per induzione una limitazione ipotizzata;
  - il **metodo dell'albero di ricorsione**, che visualizza i costi di tutte le chiamate e li somma livello per livello.
- Infine disegniamo l'albero di ricorsione di una generica equazione divide et impera: i tre comportamenti che vi osserviamo sono il punto di partenza del Master theorem, più avanti nel corso

Riepilogo

Equazioni di ricorrenza:

- il numero di passi di un algoritmo ricorsivo è definito in termini di se stesso
- una ricorrenza lo descrive

Tre metodi per risolverle:

- espansione, sostituzione, albero di ricorsione

Equazioni di ricorrenza

Introduzione: Equazioni di ricorrenza

- In questa sezione definiamo le equazioni di ricorrenza, le loro condizioni al contorno e che cosa significa risolverle
- Vediamo come una ricorrenza nasce da un algoritmo ricorsivo, e le forme più comuni di ricorrenze
- Fissiamo i dettagli tecnici che si possono trascurare e le sommatorie che usano tutti i metodi del capitolo

Definizione 1: equazione di ricorrenza

*Una **equazione di ricorrenza** (o semplicemente **ricorrenza**) è un'equazione (o una disequazione) che descrive una funzione $T : \mathbb{N} \to \mathbb{R}$ in termini dei suoi valori su input più piccoli.*

Condizioni al contorno e soluzioni

- Ogni ricorrenza ha delle **condizioni al contorno**, che danno il valore della funzione quando la dimensione dell'input è piccola, cioè quando n raggiunge (o è inferiore a) un valore specifico n_0. Precisamente, le condizioni al contorno sono

$$T(n) = g(n) \qquad \text{se } n \leq n_0,$$

  dove $g(n)$ è il valore della funzione per i valori piccoli di n
- Tipicamente $n_0 = 1$ e la condizione al contorno è $T(1) = \Theta(1)$
- **Risolvere una ricorrenza** significa determinare una forma chiusa di $T(n)$ o, almeno, il suo ordine di grandezza, cioè una funzione $h(n)$ tale che $T(n) = \Theta(h(n))$ (oppure $T(n) = O(h(n))$, oppure $T(n) = \Omega(h(n))$)

Esempio (1/2)

Consideriamo la ricorrenza

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ T(n-1) + n & \text{se } n > 1 \end{cases}$$

La condizione al contorno è $T(1) = 1$ (qui $n_0 = 1$); per $n > 1$ ogni valore si ottiene dal precedente:

- $T(1) = 1$ (condizione al contorno);
- $T(2) = T(1) + 2 = 1 + 2 = 3$;
- $T(3) = T(2) + 3 = 3 + 3 = 6$;
- $T(4) = T(3) + 4 = 6 + 4 = 10$.

Esempio (2/2)

- La ricorrenza definisce $T(n)$ per ogni $n \in \mathbb{N}$, $n \geq 1$
- Ma calcolare $T(n)$ in questo modo richiede $n - 1$ addizioni
- Con il metodo di espansione la risolveremo e otterremo la forma chiusa

$$T(n) = \frac{n(n+1)}{2}$$

Ricorrenze e algoritmi ricorsivi

- Le ricorrenze sono lo strumento naturale per determinare la complessità asintotica degli **algoritmi ricorsivi**
- In questo capitolo le ricorrenze sono oggetti matematici, e usiamo una generica funzione $T(n)$
- Quando analizziamo un algoritmo ricorsivo, la funzione descritta dalla ricorrenza è il numero di passi $S(n)$ eseguiti nel caso peggiore su un input di parametro di dimensione n (si veda il capitolo *Ricorsione*)
- Il numero di passi di una chiamata è il numero di passi eseguiti dalla chiamata stessa più il numero di passi eseguiti dalle sue chiamate ricorsive

Consideriamo il seguente algoritmo ricorsivo
che calcola la somma degli elementi di un array.

Algorithm: RecursiveSum

Input : Un array $a[1..n]$ di $n \geq 1$ numeri interi
Output: La somma $a[1] + a[2] + \cdots + a[n]$

```
1 function RecursiveSum(a, n)
2 |   if n = 1 then
3 |   |   return a[1]
4 |   end
5 |   s := RecursiveSum(a, n − 1)
6 |   return s + a[n]
7 end
```

Esegui in Colab: Python · C++ · Java

Esempio (1/2)

Sia $S(n)$ il numero di passi eseguiti da `RecursiveSum` su un array di dimensione n. Contiamo le esecuzioni di ciascuna riga.

- Se $n = 1$, le righe 2 e 3 sono eseguite una volta:

$$S(1) = 2$$

- Se $n > 1$, le righe 2, 5 e 6 sono eseguite una volta (3 passi), e inoltre la chiamata ricorsiva alla riga 5 esegue $S(n-1)$ passi

Esempio (2/2)

Quindi

$$S(n) = \begin{cases} 2 & \text{se } n = 1 \\ S(n-1) + 3 & \text{se } n > 1 \end{cases}$$

- Per esempio, $S(2) = 2 + 3 = 5$, $S(3) = 5 + 3 = 8$ e $S(4) = 8 + 3 = 11$
- Con il metodo di espansione mostreremo che

$$S(n) = 3n - 1 = \Theta(n)$$

La forma della ricorrenza rispecchia il modo in cui l'algoritmo suddivide il suo input.

Le forme più comuni sono le seguenti.

Diverse forme di ricorrenze (1/2)

- **Divide et impera.** Un algoritmo ricorsivo divide il problema in a sottoproblemi, ciascuno di dimensione n/b, li risolve ricorsivamente e ne combina le soluzioni:

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n).$$

Per esempio, l'algoritmo `MergeSort` dà luogo a $T(n) = 2T(n/2) + \Theta(n)$

- **Suddivisione non uniforme.** Un algoritmo ricorsivo può dividere il problema in sottoproblemi di dimensioni diverse. Se l'input è diviso in una parte 1/3 e una parte 2/3:

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + f(n),$$

dove $f(n)$ è il tempo richiesto per dividere il problema e per combinare le soluzioni dei sottoproblemi

Diverse forme di ricorrenze (2/2)

- **Decrementa e conquista.** I sottoproblemi non sono necessariamente una frazione costante del problema originale. Un algoritmo ricorsivo può creare un solo sottoproblema contenente un elemento in meno del problema originale (come `RecursiveSum`):

$$T(n) = T(n-1) + f(n).$$

- **Più sottoproblemi di dimensione** $n-1$. L'algoritmo che risolve il rompicapo della *Torre di Hanoi* con n dischi sposta $n-1$ dischi, poi un disco, poi di nuovo $n-1$ dischi:

$$T(n) = 2\,T(n-1) + 1,$$

la cui soluzione è esponenziale in n (la vedremo con il metodo di espansione)

Definizione 2: ricorrenza in forma standard

Una ricorrenza è in ***forma standard*** *se può essere scritta come*

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$$

dove

- n *è la dimensione dell'input (tipicamente* $n \in \mathbb{N}$*);*
- $a \geq 1$ *è il numero di sottoproblemi da risolvere;*
- $\frac{n}{b}$ *è la dimensione di ciascun sottoproblema* $(b > 1)$*;*
- $f(n)$ *è il tempo richiesto per dividere il problema e per combinare le soluzioni dei sottoproblemi.*

Forma standard con le condizioni al contorno

- Includendo le condizioni al contorno, una ricorrenza in forma standard è

$$T(n) = \begin{cases} g(1) & \text{se } n = 1 \\ a\,T\left(\frac{n}{b}\right) + f(n) & \text{se } n > 1 \end{cases}$$

dove $g(1)$ è il tempo richiesto per risolvere un sottoproblema di dimensione 1, tipicamente $g(1) = \Theta(1)$

Ricorrenze date da disuguaglianze

- Occasionalmente incontreremo ricorrenze che non sono uguaglianze ma disuguaglianze, come

$$T(n) \leq a\, T\left(\frac{n}{b}\right) + f(n).$$

Una tale ricorrenza fornisce soltanto una **limitazione superiore** per $T(n)$: la sua soluzione è data in notazione O anziché in notazione Θ

- Analogamente, se la disuguaglianza è invertita,

$$T(n) \geq a\, T\left(\frac{n}{b}\right) + f(n),$$

la ricorrenza fornisce soltanto una **limitazione inferiore** per $T(n)$, e la sua soluzione è data in notazione Ω

Metodi per risolvere le ricorrenze

In questo capitolo studiamo tre metodi per risolvere le ricorrenze:

1. Il **metodo di espansione** (detto anche di iterazione o di srotolamento): si sostituisce ripetutamente la ricorrenza in se stessa fino a raggiungere la condizione al contorno, e si calcola la sommatoria che ne risulta

2. Il **metodo di sostituzione**: si ipotizza una limitazione e poi si usa l'induzione matematica per dimostrare che l'ipotesi è corretta

3. Il **metodo dell'albero di ricorsione**: si converte la ricorrenza in un albero i cui nodi rappresentano i costi sostenuti ai vari livelli della ricorsione, e si limitano le sommatorie che ne risultano

Più avanti nel corso studiamo un quarto metodo, il **Master theorem**, che fornisce direttamente la soluzione delle ricorrenze in forma standard.

Prima di applicare i metodi fissiamo due punti tecnici:

quali dettagli di una ricorrenza si possono trascurare,
e quali sommatorie compaiono più spesso.

Aspetti tecnici: parti intere (1/2)

- Spesso trascuriamo alcuni dettagli tecnici quando enunciamo e risolviamo le ricorrenze
- Consideriamo la ricorrenza

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2\,T\left(\frac{n}{2}\right) + \Theta(n) & \text{se } n > 1 \end{cases}$$

- Quando n è dispari, i due sottoproblemi non possono avere entrambi dimensione $n/2$ (che non è un numero intero): tipicamente, hanno dimensioni $\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$

Aspetti tecnici: parti intere (2/2)

- La ricorrenza precisa è quindi

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n) & \text{se } n > 1 \end{cases}$$

- Le due ricorrenze hanno la stessa soluzione asintotica
- Poiché ci interessa l'ordine di grandezza, le parti intere inferiori e superiori si possono di solito omettere

Aspetti tecnici: potenze esatte

- Per lo stesso motivo, quando risolviamo una ricorrenza in forma standard supponiamo spesso che n sia una **potenza esatta di** b, cioè

$$n = b^k \qquad \text{per qualche } k \in \mathbb{N}$$

- In questo modo tutte le dimensioni dei sottoproblemi $n/b, n/b^2, \ldots, 1$ sono intere

- Nell'esempio precedente supponiamo $n = 2^k$

Aspetti tecnici: condizioni al contorno

- Le condizioni al contorno sono un'altra classe di dettagli che tipicamente ignoriamo
- Il tempo di esecuzione di un algoritmo su un input di dimensione costante è una costante: le ricorrenze che nascono dall'analisi degli algoritmi soddisfano in generale

$$T(n) = \Theta(1) \qquad \text{per valori piccoli di } n$$

- Per comodità, ometteremo in generale le condizioni al contorno e supporremo che $T(n)$ sia costante per n piccolo
- Cambiare il valore di $T(1)$ cambia tipicamente la soluzione esatta, ma non il suo ordine di grandezza

Semplificazioni e rigore

- Omettere parti intere, condizioni al contorno e supporre che n sia una potenza esatta di b semplifica i calcoli
- Quando è richiesta una dimostrazione rigorosa (per esempio nel metodo di sostituzione), questi dettagli devono essere trattati esplicitamente

Tutti e tre i metodi terminano con una sommatoria:
i costi delle singole chiamate vanno sommati.

Le formule seguenti sono usate in tutto il capitolo.

Osservazione 1: sommatorie e logaritmi utili (1/2)

- *La serie aritmetica: per ogni* $n \geq 1$

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

- *La serie geometrica: per ogni numero reale* $x \neq 1$ *e ogni* $k \in \mathbb{N}$

$$\sum_{i=0}^{k} x^i = 1 + x + x^2 + \cdots + x^k = \frac{x^{k+1} - 1}{x - 1}.$$

In particolare, per $x = 2$,

$$\sum_{i=0}^{k-1} 2^i = 2^k - 1.$$

Osservazione: sommatorie e logaritmi utili (2/2)

- *La serie geometrica infinita decrescente: per ogni numero reale x con $|x| < 1$*

$$\sum_{i=0}^{\infty} x^i = \frac{1}{1-x}.$$

- *Due proprietà dei logaritmi: per ogni $a, b, n > 0$ (con $b \neq 1$)*

$$a^{\log_b n} = n^{\log_b a}, \qquad \log_b n = \frac{\log_2 n}{\log_2 b}.$$

Dimostrazione (1/2)

Serie aritmetica.

- Scriviamo la somma due volte, una in ordine crescente e una in ordine decrescente, e sommiamo termine a termine
- Otteniamo n coppie, ciascuna con somma $n+1$: il doppio della somma è $n(n+1)$

Serie geometrica.

- Moltiplicando la somma per $x-1$, tutti i termini intermedi si semplificano:

$$(x-1)\sum_{i=0}^{k} x^i = \sum_{i=1}^{k+1} x^i - \sum_{i=0}^{k} x^i = x^{k+1} - 1,$$

  e dividiamo per $x-1 \neq 0$
- Per $x=2$ e $k-1$ al posto di k otteniamo $2^k - 1$

Dimostrazione (2/2)

Serie infinita.

- Se $|x| < 1$, allora $x^{k+1} \to 0$ per $k \to \infty$, e la somma finita tende a

$$\frac{0-1}{x-1} = \frac{1}{1-x}$$

Logaritmi.

- Abbiamo

$$a^{\log_b n} = \left(b^{\log_b a}\right)^{\log_b n} = \left(b^{\log_b n}\right)^{\log_b a} = n^{\log_b a}$$

- Inoltre $n = b^{\log_b n}$ dà $\log_2 n = \log_b n \cdot \log_2 b$

□

Due conseguenze

- Poiché tutti i termini sono positivi quando $0 < x < 1$, ogni somma finita $\sum_{i=0}^{k} x^i$ è minore di $\frac{1}{1-x}$

  La serie infinita dà una **limitazione superiore** per una somma geometrica finita che non dipende da k

- La seconda proprietà dei logaritmi mostra che logaritmi in basi diverse differiscono per un fattore costante

  Quindi, dentro Θ, O e Ω la base del logaritmo è irrilevante

Riepilogo (1/2)

Equazioni di ricorrenza:

- una funzione $T(n)$ descritta in termini dei suoi valori su input più piccoli
- le condizioni al contorno danno $T(n)$ per $n \leq n_0$, tipicamente $T(1) = \Theta(1)$
- risolvere una ricorrenza: forma chiusa o ordine di grandezza

Algoritmi ricorsivi:

- il numero di passi $S(n)$ soddisfa una ricorrenza, per esempio $S(n) = S(n-1) + 3$ per `RecursiveSum`
- forma standard $T(n) = a\,T(n/b) + f(n)$, con $a \geq 1$ e $b > 1$
- altre forme: $T(n-1) + f(n)$, $2\,T(n-1) + 1$, $T(n/3) + T(2n/3) + f(n)$

Riepilogo (2/2)

Aspetti tecnici:

- parti intere e condizioni al contorno si possono di solito omettere
- supponiamo spesso che n sia una potenza esatta di b
- una dimostrazione rigorosa deve trattarli esplicitamente

Sommatorie utili:

- serie aritmetica, serie geometrica finita e infinita
- $a^{\log_b n} = n^{\log_b a}$; la base del logaritmo è irrilevante dentro Θ

Il metodo di espansione

Introduzione: Il metodo di espansione

- Il modo più diretto per risolvere una ricorrenza è applicarla ripetutamente
- Ogni applicazione sostituisce un valore di T con un valore su un argomento più piccolo, e dopo abbastanza applicazioni resta soltanto la condizione al contorno
- Risolviamo così cinque ricorrenze, da tempi di esecuzione lineari fino a esponenziali

Il metodo di espansione

Nel **metodo di espansione** (detto anche **iterazione** o **srotolamento**):

1. si sostituisce la ricorrenza in se stessa alcune volte, fino a riconoscere lo schema generale dopo k sostituzioni;
2. si sceglie k in modo che l'argomento raggiunga la condizione al contorno;
3. si calcola (o si limita) la sommatoria che ne risulta.

- Il metodo fornisce una forma chiusa esatta quando la sommatoria si può calcolare
- Lo schema riconosciuto al passo 1 si può dimostrare rigorosamente per induzione su k

Cominciamo con le ricorrenze di tipo decrementa e conquista,

$$T(n-1)+f(n),$$

e poi passiamo a quelle di tipo divide et impera,

$$a\,T(n/2)+f(n).$$

Esempio: $T(n) = T(n-1) + c$ (1/2)

Siano $c, d > 0$ costanti e

$$T(n) = \begin{cases} d & \text{se } n = 1 \\ T(n-1) + c & \text{se } n > 1 \end{cases}$$

Passo 1 (espansione).

$$\begin{aligned} T(n) &= T(n-1) + c \\ &= \big(T(n-2) + c\big) + c = T(n-2) + 2c \\ &= \big(T(n-3) + c\big) + 2c = T(n-3) + 3c \\ &\vdots \\ &= T(n-k) + k\,c. \end{aligned}$$

Esempio: $T(n) = T(n-1) + c$ (2/2)

Passo 2 (condizione al contorno).

- La condizione al contorno è raggiunta quando $n - k = 1$, cioè $k = n - 1$

Passo 3 (somma).

$$T(n) = T(1) + (n-1)\,c = c\,n + (d - c) = \Theta(n).$$

- Per `RecursiveSum` abbiamo $c = 3$ e $d = 2$, quindi

$$S(n) = 3(n-1) + 2 = 3n - 1 = \Theta(n)$$

- Questo dà $S(2) = 5$, $S(3) = 8$, $S(4) = 11$, come calcolato direttamente dalla ricorrenza

Esempio: $T(n) = T(n-1) + n$ (1/3)

Sia

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ T(n-1) + n & \text{se } n > 1 \end{cases}$$

Passo 1 (espansione).

$$\begin{aligned} T(n) &= T(n-1) + n \\ &= T(n-2) + (n-1) + n \\ &= T(n-3) + (n-2) + (n-1) + n \\ &\vdots \\ &= T(n-k) + (n-k+1) + \cdots + (n-1) + n. \end{aligned}$$

Esempio: $T(n) = T(n-1) + n\ (2/3)$

Passo 2 (condizione al contorno).

- $n - k = 1$, cioè $k = n - 1$

Passo 3 (somma).

- Usando la serie aritmetica,

$$\begin{aligned} T(n) &= T(1) + 2 + 3 + \cdots + n \\ &= 1 + 2 + \cdots + n \\ &= \frac{n(n+1)}{2} = \Theta(n^2) \end{aligned}$$

Esempio: $T(n) = T(n-1) + n$ (3/3)

- Infatti,

$$\lim_{n\to\infty} \frac{n(n+1)/2}{n^2} = \lim_{n\to\infty} \frac{1}{2}\left(1 + \frac{1}{n}\right) = \frac{1}{2} \in \mathbb{R}_{>0}$$

- La forma chiusa dà $T(1) = 1$, $T(2) = 3$, $T(3) = 6$, $T(4) = 10$, i valori calcolati all'inizio del capitolo

Esempio: $T(n) = 2T(n-1) + 1$, la Torre di Hanoi (1/3)

Sia

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 2\,T(n-1) + 1 & \text{se } n > 1 \end{cases}$$

Passo 1 (espansione).

$$\begin{aligned} T(n) &= 2\,T(n-1) + 1 \\ &= 2\big(2\,T(n-2) + 1\big) + 1 = 2^2\,T(n-2) + 2 + 1 \\ &= 2^2\big(2\,T(n-3) + 1\big) + 2 + 1 = 2^3\,T(n-3) + 2^2 + 2 + 1 \\ &\vdots \\ &= 2^k\,T(n-k) + \sum_{i=0}^{k-1} 2^i. \end{aligned}$$

Esempio: $T(n) = 2T(n-1) + 1$, la Torre di Hanoi (2/3)

Passo 2 (condizione al contorno).

- $n - k = 1$, cioè $k = n - 1$

Passo 3 (somma).

- Usando $\sum_{i=0}^{k-1} 2^i = 2^k - 1$,

$$\begin{aligned} T(n) &= 2^{n-1}\, T(1) + \left(2^{n-1} - 1\right) \\ &= 2^{n-1} + 2^{n-1} - 1 \\ &= 2^n - 1 = \Theta(2^n) \end{aligned}$$

Esempio: $T(n) = 2T(n-1) + 1$, la Torre di Hanoi (3/3)

- Questo è il numero di mosse necessarie per risolvere il rompicapo della Torre di Hanoi con n dischi: il tempo di esecuzione è **esponenziale** in n
- La funzione `hanoi` del codice del corso conta le sue mosse; eseguendola otteniamo

n	1	2	3	4	5	6	7	8	10
mosse	1	3	7	15	31	63	127	255	1023

cioè esattamente $2^n - 1$ mosse

Le due ricorrenze seguenti dimezzano la dimensione del problema a ogni chiamata.

Per evitare le parti intere supponiamo che n sia una potenza esatta di 2.

Esempio: $T(n) = T(n/2) + c$ (1/2)

Siano $c, d > 0$ costanti, sia $n = 2^k$ una potenza esatta di 2, e

$$T(n) = \begin{cases} d & \text{se } n = 1 \\ T\left(\frac{n}{2}\right) + c & \text{se } n > 1 \end{cases}$$

Passo 1 (espansione).

$$T(n) = T\left(\frac{n}{2}\right) + c = T\left(\frac{n}{2^2}\right) + 2c = T\left(\frac{n}{2^3}\right) + 3c$$
$$= \cdots = T\left(\frac{n}{2^j}\right) + j\,c.$$

Esempio: $T(n) = T(n/2) + c$ (2/2)

Passo 2 (condizione al contorno).

- $\frac{n}{2^j} = 1$, cioè $j = \log_2 n$

Passo 3 (somma).

$$T(n) = T(1) + c \log_2 n = c \log_2 n + d = \Theta(\log_2 n).$$

- Questa ricorrenza descrive, per esempio, la versione ricorsiva di `BinarySearch`: a ogni chiamata la dimensione del sotto-array è dimezzata

Esempio: $T(n) = 2T(n/2) + n$ (1/3)

Sia $n = 2^k$ una potenza esatta di 2 e

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 2\,T\left(\frac{n}{2}\right) + n & \text{se } n > 1 \end{cases}$$

La espandiamo come negli esempi precedenti.

Esempio: $T(n) = 2T(n/2) + n$ (2/3)

Passo 1 (espansione).

$$
\begin{aligned}
T(n) &= 2\,T\left(\frac{n}{2}\right) + n \\
&= 2\left(2\,T\left(\frac{n}{2^2}\right) + \frac{n}{2}\right) + n \\
&= 2^2\,T\left(\frac{n}{2^2}\right) + 2n \\
&= 2^2\left(2\,T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}\right) + 2n \\
&= 2^3\,T\left(\frac{n}{2^3}\right) + 3n \\
&\vdots \\
&= 2^j\,T\left(\frac{n}{2^j}\right) + j\,n.
\end{aligned}
$$

Esempio: $T(n) = 2T(n/2) + n$ (3/3)

Passo 2 (condizione al contorno).

- $j = \log_2 n$, in modo che $2^j = n$

Passo 3 (somma).

$$T(n) = n\,T(1) + n\log_2 n = n\log_2 n + n = \Theta(n\log_2 n).$$

Verifica:

- dalla ricorrenza $T(2) = 2 + 2 = 4$, $T(4) = 8 + 4 = 12$, $T(8) = 24 + 8 = 32$
- dalla forma chiusa $T(8) = 8 \cdot 3 + 8 = 32$

Quando il metodo di espansione funziona bene

- Il metodo di espansione funziona bene quando la ricorrenza ha un **solo termine ricorsivo**
- Con più termini ricorsivi di dimensioni diverse (per esempio $T(n/3) + T(2n/3)$) l'espansione diventa rapidamente ingestibile
- In questi casi il metodo dell'albero di ricorsione è più conveniente

Riepilogo

Il metodo di espansione:

- espandere, raggiungere la condizione al contorno, calcolare la sommatoria
- fornisce la forma chiusa esatta quando la sommatoria si può calcolare

Soluzioni trovate:

- $T(n-1)+c$: $\Theta(n)$ $\quad$ $T(n-1)+n$: $\frac{n(n+1)}{2} = \Theta(n^2)$
- $2T(n-1)+1$: $2^n - 1 = \Theta(2^n)$ (Torre di Hanoi)
- $T(n/2)+c$: $\Theta(\log_2 n)$ $\quad$ $2T(n/2)+n$: $n \log_2 n + n = \Theta(n \log_2 n)$

Limite: un solo termine ricorsivo

Il metodo di sostituzione

Introduzione: Il metodo di sostituzione

- Il metodo di espansione richiede dimensioni esatte dei sottoproblemi: con le parti intere lo schema non è più pulito
- Quando abbiamo già un'idea della soluzione, un approccio più robusto è *dimostrarla* per induzione
- Dimostriamo una limitazione superiore e una inferiore per una ricorrenza con le parti intere, vediamo come rafforzare un'ipotesi e come aiuta un cambio di variabile

Il metodo di sostituzione

Il **metodo di sostituzione** comprende due passi:

1. **ipotizzare** la forma della soluzione (per esempio $T(n) \leq c\, n \log_2 n$, con c una costante incognita);
2. usare l'**induzione matematica** per trovare le costanti e mostrare che la soluzione funziona.

Osservazioni sul metodo di sostituzione

- Il nome deriva dal fatto che, nel passo induttivo, *sostituiamo* la limitazione ipotizzata ai valori della funzione sugli input più piccoli
- Il metodo è molto potente, ma richiede una buona ipotesi. L'ipotesi può venire dall'esperienza (ricorrenze simili), dal metodo di espansione o da un albero di ricorsione
- Possiamo usare il metodo di sostituzione per dimostrare separatamente limitazioni superiori (O) e inferiori (Ω). Insieme danno Θ
- Poiché la notazione asintotica richiede la limitazione solo per $n \geq n_0$, siamo liberi di scegliere n_0: questo è utile quando la limitazione non vale per i valori più piccoli di n

Un esempio con le parti intere: limitazione superiore

- Consideriamo la ricorrenza

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 2\,T\left(\lfloor n/2 \rfloor\right) + n & \text{se } n > 1 \end{cases}$$

- Con il metodo di espansione (dove n è una potenza di 2) abbiamo trovato $T(n) = n \log_2 n + n$, quindi ipotizziamo $T(n) = O(n \log_2 n)$
- Vogliamo dimostrare che esistono costanti $c > 0$ e n_0 tali che

$$T(n) \leq c\, n \log_2 n \qquad \text{per ogni } n \geq n_0$$

Limitazione superiore: passo induttivo (1/2)

- Supponiamo che la limitazione valga per tutti i valori m con $n_0 \leq m < n$
- In particolare per $m = \lfloor n/2 \rfloor$ (questo richiede $\lfloor n/2 \rfloor \geq n_0$):

$$T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \log_2 \lfloor n/2 \rfloor$$

Limitazione superiore: passo induttivo (2/2)

Sostituendo nella ricorrenza:

$$
\begin{aligned}
T(n) &\leq 2\,c\,\lfloor n/2 \rfloor \log_2 \lfloor n/2 \rfloor + n \\
&\leq 2\,c\,\frac{n}{2}\log_2 \frac{n}{2} + n && \text{(poiché } \lfloor n/2 \rfloor \leq n/2\text{)} \\
&= c\,n\log_2 n - c\,n\log_2 2 + n \\
&= c\,n\log_2 n - c\,n + n \\
&\leq c\,n\log_2 n,
\end{aligned}
$$

dove l'ultimo passaggio vale purché $c \geq 1$.

Limitazione superiore: casi base (1/2)

- L'induzione richiede che la limitazione valga per le condizioni al contorno. Tuttavia, per $n = 1$:

$$T(1) = 1 > c \cdot 1 \cdot \log_2 1 = 0,$$

per ogni c. Quindi la limitazione *non vale* per $n = 1$

- Sfruttiamo la libertà di scegliere n_0: dimostriamo la limitazione per $n \geq n_0 = 2$
- Per $n \geq 4$ si ha $\lfloor n/2 \rfloor \geq 2$, quindi il passo induttivo usa soltanto valori di T su argomenti ≥ 2
- Per $n = 2$ e $n = 3$ si ha $\lfloor n/2 \rfloor = 1$, quindi il passo induttivo userebbe $T(1)$, che non soddisfa la limitazione. Pertanto $n = 2$ e $n = 3$ devono essere trattati come **casi base**

Limitazione superiore: casi base (2/2)

- Dalla ricorrenza,

$$T(2) = 2\,T(1) + 2 = 4 \qquad \text{e} \qquad T(3) = 2\,T(1) + 3 = 5$$

- Ci serve

$$4 \leq c \cdot 2 \log_2 2 = 2c \qquad \text{e} \qquad 5 \leq c \cdot 3 \log_2 3$$

- Entrambe valgono per $c \geq 2$ (poiché $3 \log_2 3 \approx 4.75$, si ha $2 \cdot 3 \log_2 3 \approx 9.51 \geq 5$)

Osservazione 2: limitazione superiore per $2T(\lfloor n/2 \rfloor) + n$

La funzione

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 2\,T\left(\lfloor n/2 \rfloor\right) + n & \text{se } n > 1 \end{cases}$$

soddisfa $T(n) \leq 2\,n \log_2 n$ *per ogni* $n \geq 2$. *Quindi* $T(n) = O(n \log_2 n)$.

Dimostrazione

Sia $c = 2$. Procediamo per induzione forte su $n \geq 2$.

Casi base $n = 2, 3$: $T(2) = 4 \leq 2 \cdot 2 \cdot \log_2 2 = 4$ e $T(3) = 5 \leq 2 \cdot 3 \log_2 3 \approx 9.51$.

Passo induttivo $n \geq 4$: abbiamo $2 \leq \lfloor n/2 \rfloor < n$, quindi per l'ipotesi induttiva $T(\lfloor n/2 \rfloor) \leq 2\lfloor n/2 \rfloor \log_2 \lfloor n/2 \rfloor$. Allora

$$\begin{aligned} T(n) &\leq 4\lfloor n/2 \rfloor \log_2 \lfloor n/2 \rfloor + n \\ &\leq 2n \log_2 \frac{n}{2} + n \\ &= 2n \log_2 n - 2n + n \\ &\leq 2n \log_2 n. \end{aligned}$$

□

Un esempio con le parti intere: limitazione inferiore

- Dimostriamo ora $T(n) = \Omega(n \log_2 n)$ per la stessa ricorrenza. Cerchiamo una costante $d > 0$ tale che $T(n) \geq d\, n \log_2 n$ per ogni $n \geq 1$
- Per la limitazione inferiore, la parte intera lavora contro di noi: servono limitazioni inferiori per $\lfloor n/2 \rfloor$. Per ogni $n \geq 2$:

$$2\lfloor n/2 \rfloor \geq n - 1 \qquad \text{e} \qquad \lfloor n/2 \rfloor \geq \frac{n}{3}$$

- La prima vale perché $\lfloor n/2 \rfloor \geq (n-1)/2$
- La seconda vale per $n = 2$ ($1 \geq 2/3$) e, per $n \geq 3$, perché $(n-1)/2 \geq n/3$ è equivalente a $n \geq 3$

Limitazione inferiore: passo induttivo (1/2)

- Sia $n \geq 2$ e supponiamo $T(\lfloor n/2 \rfloor) \geq d \lfloor n/2 \rfloor \log_2 \lfloor n/2 \rfloor$
- Poiché $\lfloor n/2 \rfloor \geq 1$, il fattore $\log_2 \lfloor n/2 \rfloor$ è non negativo, e

$$
\begin{aligned}
T(n) &\geq 2\, d\, \lfloor n/2 \rfloor \log_2 \lfloor n/2 \rfloor + n \\
&\geq d\,(n-1) \log_2 \frac{n}{3} + n \\
&= d\, n \log_2 n - d\, n \log_2 3 - d \log_2 n + d \log_2 3 + n \\
&\geq d\, n \log_2 n + \Big(n - d\, n \log_2 3 - d \log_2 n \Big).
\end{aligned}
$$

Limitazione inferiore: passo induttivo (2/2)

- Poiché $\log_2 n \leq n$, il termine fra parentesi è almeno

$$n(1 - d\log_2 3 - d),$$

che è non negativo se

$$d \leq \frac{1}{1 + \log_2 3} \approx 0.387$$

- Scegliamo $d = \frac{1}{4}$

Caso base ($n = 1$):

$$T(1) = 1 \geq \frac{1}{4} \cdot 1 \cdot \log_2 1 = 0. \checkmark$$

Osservazione 3: limitazione inferiore per $2T(\lfloor n/2 \rfloor) + n$

La funzione

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 2\,T\left(\lfloor n/2 \rfloor\right) + n & \text{se } n > 1 \end{cases}$$

soddisfa $T(n) \geq \frac{1}{4}\, n \log_2 n$ *per ogni* $n \geq 1$. *Quindi* $T(n) = \Omega(n \log_2 n)$.

Dimostrazione

Per induzione forte su $n \geq 1$, con $d = \frac{1}{4}$.

- Per $n = 1$, $T(1) = 1 \geq 0$
- Per $n \geq 2$ abbiamo $1 \leq \lfloor n/2 \rfloor < n$, quindi l'ipotesi induttiva vale per $\lfloor n/2 \rfloor$, e il calcolo del passo induttivo precedente dà

$$\begin{aligned} T(n) &\geq \tfrac{1}{4}\, n \log_2 n + n \left(1 - \tfrac{1}{4} \log_2 3 - \tfrac{1}{4}\right) \\ &\geq \tfrac{1}{4}\, n \log_2 n, \end{aligned}$$

poiché $1 - \frac{1}{4}(1 + \log_2 3) \approx 0.35 > 0$

□

La soluzione di $2T(\lfloor n/2 \rfloor) + n$

- Dalle due Osservazioni precedenti, la soluzione della ricorrenza è

$$T(n) = \Theta(n \log_2 n)$$

- La parte intera non cambia l'ordine di grandezza trovato per n potenza di 2

- Numericamente, per ogni n con $2 \leq n < 200\,000$ il rapporto

$$\frac{T(n)}{n \log_2 n}$$

è compreso tra 0.83 e 2

A volte l'ipotesi è corretta,
ma l'induzione non funziona
perché l'ipotesi induttiva *non è abbastanza forte*.

Quando l'ipotesi va rafforzata (1/2)

- Consideriamo

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 2\,T(\lfloor n/2 \rfloor) + 1 & \text{se } n > 1 \end{cases}$$

- Per n potenza di 2 si trova facilmente $T(n) = 2n - 1$ (per esempio $T(2) = 3$, $T(4) = 7$, $T(8) = 15$)
- Quindi ipotizziamo $T(n) = O(n)$, cioè $T(n) \leq c\,n$

Quando l'ipotesi va rafforzata (2/2)

- **Tentativo ingenuo.** Sostituendo l'ipotesi,

$$T(n) \leq 2\,c\,\lfloor n/2 \rfloor + 1 \leq c\,n + 1,$$

che non implica $T(n) \leq c\,n$ per nessuna scelta di c

L'induzione fallisce, per un termine di ordine inferiore (la costante 1)

- **Ipotesi rafforzata.** È allettante provare un'ipotesi più grande, come $c\,n^2$, ma non serve

Invece *sottraiamo* un termine di ordine inferiore: ipotizziamo

$$T(n) \leq c\,n - d,$$

con $d \geq 0$ una costante

L'ipotesi rafforzata funziona

- Sostituendo l'ipotesi rafforzata,

$$T(n) \leq 2\big(c\,\lfloor n/2 \rfloor - d\big) + 1 \leq c\,n - 2d + 1 \leq c\,n - d,$$

dove l'ultimo passaggio vale purché $d \geq 1$

- **Caso base.** $T(1) = 1 \leq c - d$: con $d = 1$ serve $c \geq 2$

Osservazione 4: ipotesi rafforzata per $2T(\lfloor n/2 \rfloor) + 1$

La funzione

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 2\,T\left(\lfloor n/2 \rfloor\right) + 1 & \text{se } n > 1 \end{cases}$$

soddisfa $n \leq T(n) \leq 2n - 1$ *per ogni* $n \geq 1$. *Quindi* $T(n) = \Theta(n)$.

Dimostrazione

Per induzione forte su n.

- Per $n = 1$: $1 \leq T(1) = 1 \leq 2 \cdot 1 - 1$
- Per $n \geq 2$, abbiamo $1 \leq \lfloor n/2 \rfloor < n$ e, per l'ipotesi induttiva,

$$\lfloor n/2 \rfloor \leq T(\lfloor n/2 \rfloor) \leq 2\lfloor n/2 \rfloor - 1$$

- *Limitazione superiore*:
  $T(n) \leq 2\big(2\lfloor n/2 \rfloor - 1\big) + 1 = 4\lfloor n/2 \rfloor - 1 \leq 2n - 1$
- *Limitazione inferiore*: $T(n) \geq 2\lfloor n/2 \rfloor + 1 \geq (n-1) + 1 = n$

□

Sottrarre un termine di ordine inferiore

- Quando l'induzione fallisce soltanto a causa di un **termine di ordine inferiore**, non aumentare l'ipotesi
- Invece, **sottrai un termine di ordine inferiore** da essa:
  - $c\,n - d$ invece di $c\,n$;
  - $c\,n^2 - d\,n$ invece di $c\,n^2$
- L'ipotesi più forte fa funzionare l'induzione

Evitare le trappole

- Nel tentativo ingenuo sarebbe sbagliato concludere

$$\text{“}T(n) \leq c\,n + 1 = O(n)\text{, fatto”}$$

- L'induzione deve dimostrare *esattamente* la forma dell'ipotesi induttiva, $T(n) \leq c\,n$, con la *stessa* costante c

- Altrimenti la costante crescerebbe a ogni passo della ricorsione

Cambio di variabile

A volte una manipolazione algebrica,
come un cambio di variabile,
trasforma una ricorrenza sconosciuta in una nota,
la cui soluzione può allora essere riutilizzata.

Esempio: $T(n) = 2T(\lfloor\sqrt{n}\rfloor) + \log_2 n$ (1/3)

Consideriamo

$$T(n) = 2\,T\left(\lfloor\sqrt{n}\rfloor\right) + \log_2 n,$$

con condizione al contorno $T(2) = 1$.

Passo 1 (ridenominazione).

- Ignoriamo l'arrotondamento, cioè supponiamo che $n = 2^{2^k}$, in modo che tutte le radici quadrate siano potenze esatte di 2
- Poniamo $m = \log_2 n$, cioè $n = 2^m$ e $\sqrt{n} = 2^{m/2}$:

$$T(2^m) = 2\,T\left(2^{m/2}\right) + m$$

Esempio: $T(n) = 2T(\lfloor\sqrt{n}\rfloor) + \log_2 n$ (2/3)

Passo 2 (nuova funzione).

- Definiamo $R(m) = T(2^m)$ e otteniamo

$$R(m) = 2\,R\left(\frac{m}{2}\right) + m,$$

che è la ricorrenza $2T(n/2) + n$ risolta con il metodo di espansione

- Poiché $T(2) = 1$, si ha $R(1) = 1$, e quindi

$$R(m) = m \log_2 m + m$$

Esempio: $T(n) = 2T(\lfloor\sqrt{n}\rfloor) + \log_2 n$ (3/3)

Passo 3 (ritorno a T).

- Con $m = \log_2 n$:

$$\begin{aligned} T(n) &= R(\log_2 n) \\ &= \log_2 n \cdot \log_2 \log_2 n + \log_2 n \\ &= \Theta(\log_2 n \cdot \log_2 \log_2 n) \end{aligned}$$

Verifica:

- $T(4) = 2\,T(2) + 2 = 4$ e $R(2) = 2 \cdot 1 + 2 = 4$
- $T(16) = 2\,T(4) + 4 = 12$ e $R(4) = 4 \cdot 2 + 4 = 12$

Riepilogo (1/2)

Il metodo di sostituzione:

- ipotizzare la forma della soluzione, poi dimostrarla per induzione
- dimostrare O e Ω separatamente; insieme danno Θ
- la libertà di scegliere n_0 permette di trattare i casi base

La ricorrenza $2T(\lfloor n/2 \rfloor) + n$:

- $T(n) \leq 2\, n \log_2 n$ per $n \geq 2$, con casi base $n = 2, 3$
- $T(n) \geq \frac{1}{4}\, n \log_2 n$ per $n \geq 1$
- quindi $T(n) = \Theta(n \log_2 n)$: la parte intera non cambia l'ordine di grandezza

Riepilogo (2/2)

Rafforzare l'ipotesi:

- per $2T(\lfloor n/2 \rfloor) + 1$ l'ipotesi $c\,n$ fallisce per un termine di ordine inferiore
- l'ipotesi $c\,n - d$ funziona: $n \leq T(n) \leq 2n - 1$
- l'induzione deve dimostrare esattamente l'ipotesi induttiva, con la stessa costante

Cambio di variabile:

- $m = \log_2 n$ trasforma $2T(\lfloor \sqrt{n} \rfloor) + \log_2 n$ in $2R(m/2) + m$
- soluzione $\Theta(\log_2 n \cdot \log_2 \log_2 n)$

Il metodo dell'albero di ricorsione

Introduzione: Il metodo dell'albero di ricorsione

- Il metodo di sostituzione dimostra un'ipotesi, ma non la produce
- L'albero di ricorsione è lo strumento che produce buone ipotesi, e in molti casi fornisce direttamente la soluzione
- Lo applichiamo a quattro ricorrenze: costi per livello che decrescono, restano costanti o crescono, e un albero sbilanciato

Alberi di ricorsione

- In un albero di ricorsione, ogni nodo rappresenta il costo di un singolo sottoproblema da qualche parte nell'insieme delle invocazioni ricorsive della funzione. I figli di un nodo sono i sottoproblemi da esso generati
- Può essere usato per risolvere le ricorrenze, cioè per determinarne la complessità asintotica, oppure per produrre una buona ipotesi da verificare con il metodo di sostituzione
- Idea principale: sommiamo i costi all'interno di ciascun livello dell'albero per ottenere un insieme di costi per livello, e poi sommiamo tutti i costi per livello per determinare il costo totale di tutti i livelli della ricorsione

Il metodo dell'albero di ricorsione

Nel **metodo dell'albero di ricorsione** determiniamo:

1. la dimensione dei sottoproblemi e il numero di nodi a ciascuna profondità i;
2. il costo di ciascun livello (numero di nodi $\times$ costo di ciascun nodo);
3. la profondità dell'albero (quando la dimensione del sottoproblema raggiunge la condizione al contorno) e il costo delle foglie;
4. la somma dei costi di tutti i livelli.

- Applichiamo il metodo a quattro ricorrenze, che mostrano i tre possibili comportamenti dei costi per livello (decrescente, costante, crescente) e il caso di un albero sbilanciato

Costi decrescenti per livello: $T(n) = 3T(n/4) + c\,n^2$

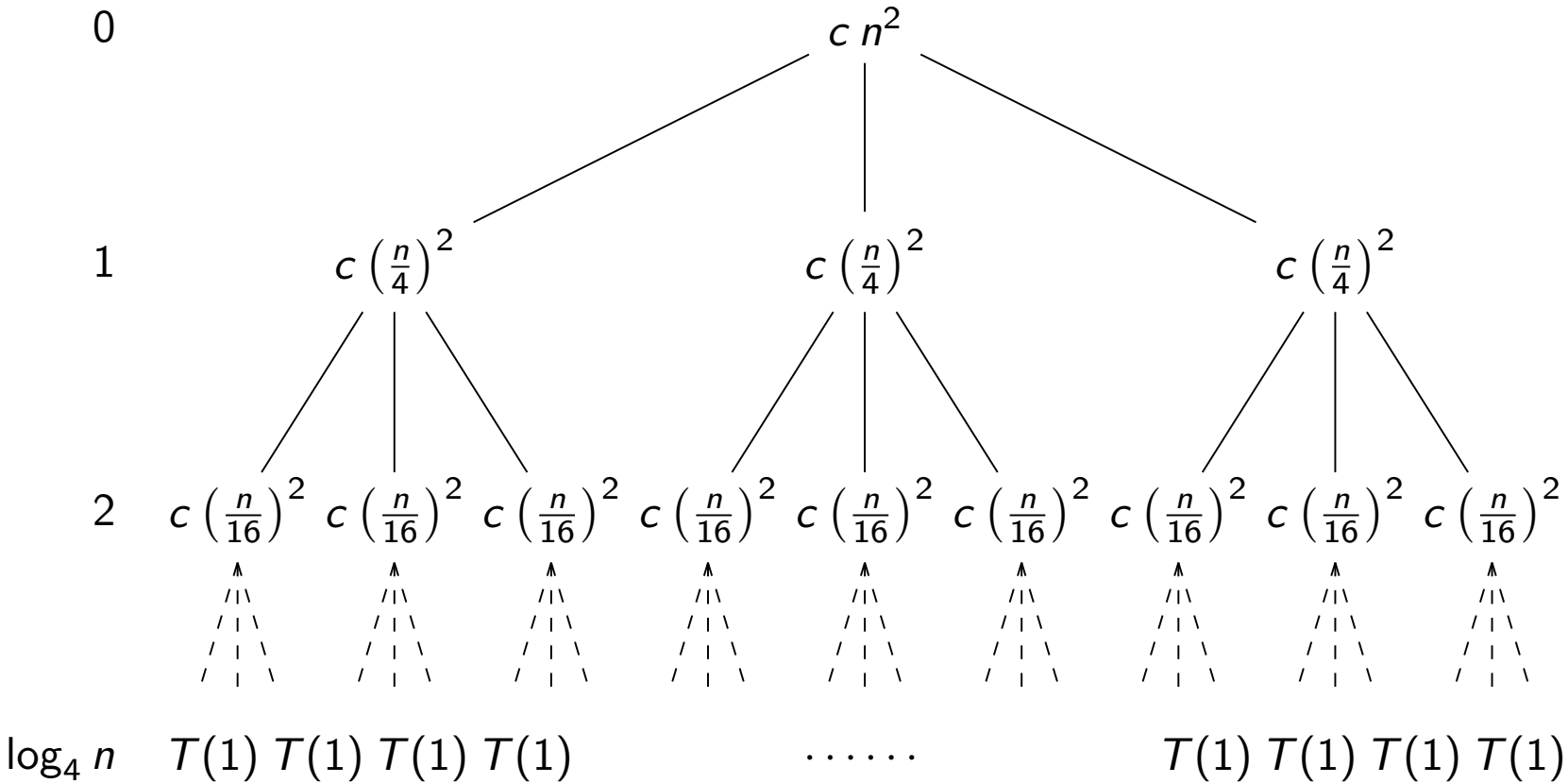
- Consideriamo la ricorrenza ($c > 0$ una costante)

$$T(n) = 3\,T\left(\frac{n}{4}\right) + c\,n^2$$

- Per comodità supponiamo che n sia una potenza esatta di 4, in modo che tutte le dimensioni dei sottoproblemi siano intere

- Nell'albero di ricorsione ogni nodo ha tre figli, la dimensione del sottoproblema è divisa per 4 a ogni livello, e le foglie si trovano a profondità $\log_4 n$

Albero di ricorsione di $T(n) = 3T(n/4) + c\,n^2$ (1/2)



Albero di ricorsione di $T(n) = 3T(n/4) + c\,n^2$ (2/2)

Il costo di ciascun livello dell'albero:

profondità	**costo del livello**
0	$c\,n^2$
1	$\frac{3}{16}\,c\,n^2$
2	$\left(\frac{3}{16}\right)^2 c\,n^2$
$\vdots$	$\vdots$
$\log_4 n$	$\Theta\left(n^{\log_4 3}\right)$

Leggere l'albero di ricorsione

- Il termine $c\,n^2$ alla radice rappresenta il costo al livello più alto della ricorsione (divisione e combinazione)
- I tre sottoalberi della radice rappresentano i costi sostenuti dai sottoproblemi di dimensione $\frac{n}{4}$; alla profondità successiva la dimensione diventa $\frac{n}{16}$, e così via
- Le foglie rappresentano i punti in cui la ricorsione "tocca il fondo", cioè in cui si raggiunge il *caso base* della ricorsione

Esempio: risolvere $T(n) = 3T(n/4) + c\,n^2$ (1/6)

Passo 1 (dimensioni e nodi).

- La dimensione del sottoproblema di ciascun nodo a profondità i è $\frac{n}{4^i}$
- Ogni livello ha 3 volte più nodi del livello precedente, quindi il numero di nodi a profondità i è 3^i

Passo 2 (costo di un livello).

- Ogni nodo a profondità i ($i = 0, 1, 2, \ldots, \log_4 n - 1$) ha un costo di $c\left(\frac{n}{4^i}\right)^2$
- Moltiplicando, il costo complessivo di tutti i nodi a profondità i è

$$3^i\, c \left(\frac{n}{4^i}\right)^2 = \left(\frac{3}{16}\right)^i c\, n^2$$

Esempio: risolvere $T(n) = 3T(n/4) + c\,n^2$ (2/6)

Passo 3 (profondità e foglie).

- La dimensione del sottoproblema raggiunge 1 quando $\frac{n}{4^i} = 1$, cioè quando $i = \log_4 n$
- Di conseguenza, l'albero ha $\log_4 n + 1$ livelli (alle profondità $0, 1, 2, \ldots, \log_4 n$)
- Il livello più basso ha, usando $a^{\log_b n} = n^{\log_b a}$,

$$3^{\log_4 n} = n^{\log_4 3}$$

nodi (le foglie dell'albero), ciascuno con un costo costante $T(1)$

- Il costo totale del livello più basso è

$$n^{\log_4 3}\, T(1) = \Theta\left(n^{\log_4 3}\right)$$

Esempio: risolvere $T(n) = 3T(n/4) + c\,n^2$ (3/6)

Passo 4 (somma). Sommando i costi di tutti i livelli otteniamo

$$\begin{aligned} T(n) &= c\,n^2 + \frac{3}{16}\,c\,n^2 + \left(\frac{3}{16}\right)^2 c\,n^2 + \cdots \\ &\quad + \left(\frac{3}{16}\right)^{\log_4 n - 1} c\,n^2 + \Theta\left(n^{\log_4 3}\right) \\ &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i c\,n^2 + \Theta\left(n^{\log_4 3}\right). \end{aligned}$$

Usando la serie geometrica otteniamo il valore esatto

$$T(n) = \frac{16}{13}\left(1 - \left(\frac{3}{16}\right)^{\log_4 n}\right) c\,n^2 + \Theta\left(n^{\log_4 3}\right).$$

Esempio: risolvere $T(n) = 3T(n/4) + c\,n^2$ (4/6)

Limitazione superiore.

- È più semplice limitare la somma finita con la serie geometrica infinita decrescente:

$$\begin{aligned} T(n) &< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i c\,n^2 + \Theta\left(n^{\log_4 3}\right) \\ &= \frac{1}{1-\frac{3}{16}}\, c\,n^2 + \Theta\left(n^{\log_4 3}\right) \\ &= \frac{16}{13}\, c\,n^2 + \Theta\left(n^{\log_4 3}\right) \\ &= O(n^2), \end{aligned}$$

poiché $\log_4 3 \approx 0.79 < 2$, quindi $n^{\log_4 3} = O(n^2)$.

Esempio: risolvere $T(n) = 3T(n/4) + c\,n^2$ (5/6)

Limitazione inferiore.

- Tutti i termini della somma sono non negativi, e la sola radice costa $c\,n^2$
- Quindi

$$T(n) \geq c\,n^2 = \Omega(n^2)$$

Esempio: risolvere $T(n) = 3T(n/4) + c\,n^2$ (6/6)

Verifica numerica ($c = 1,\ T(1) = 1$):

- La ricorrenza dà

$$T(4) = 19, \qquad T(16) = 313, \qquad T(64) = 5035,$$

i valori della formula esatta qui sopra

- I rapporti sono

$$\frac{T(n)}{n^2} = 1.1875,\ 1.2227,\ 1.2292,$$

che si avvicinano a $\frac{16}{13} \approx 1.2308$

Costi decrescenti per livello: domina la radice

- La soluzione di $T(n) = 3T(n/4) + c\,n^2$ è

$$T(n) = \Theta(n^2)$$

- I costi dei livelli decrescono geometricamente
- Il costo totale è dominato dal costo della **radice**

Costi costanti per livello: $T(n) = 2T(n/2) + c\,n$

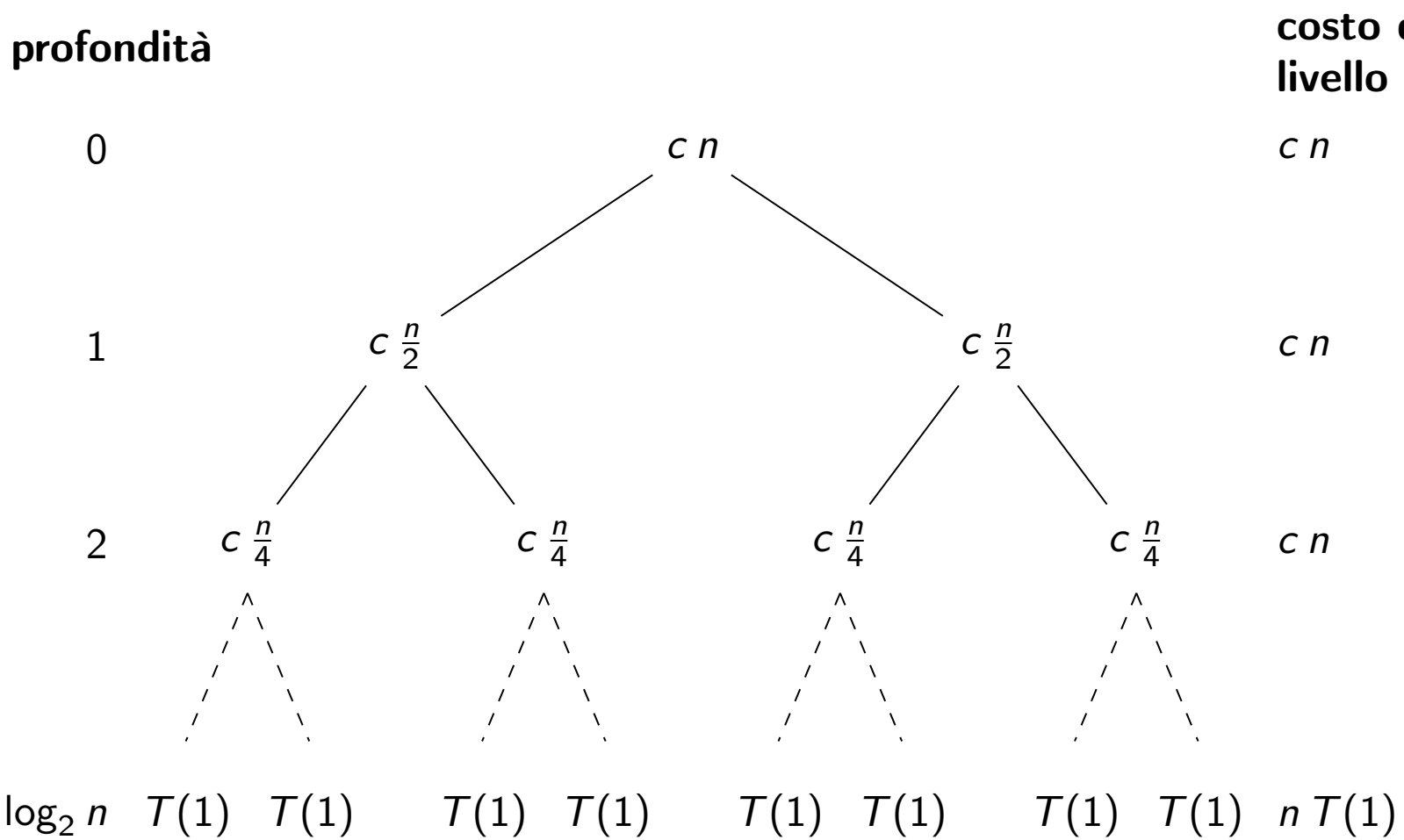
- Consideriamo la ricorrenza ($c > 0$ una costante), che descrive algoritmi come `MergeSort`:

$$T(n) = 2\,T\left(\frac{n}{2}\right) + c\,n,$$

  dove n è una potenza esatta di 2

- Nel suo albero di ricorsione ogni livello ha costo totale $c\,n$

Albero di ricorsione di $T(n) = 2T(n/2) + c\,n$



Esempio: risolvere $T(n) = 2T(n/2) + c\,n$ (1/2)

Passi 1 e 2.

- A profondità i ci sono 2^i nodi, ciascuno corrispondente a un sottoproblema di dimensione $\frac{n}{2^i}$ e con costo $c\,\frac{n}{2^i}$
- Il costo di ciascun livello è quindi

$$2^i\, c\, \frac{n}{2^i} = c\, n$$

Passo 3.

- La dimensione del sottoproblema raggiunge 1 a profondità $\log_2 n$
- Il livello più basso ha $2^{\log_2 n} = n$ foglie, con costo totale $n\,T(1) = \Theta(n)$

Esempio: risolvere $T(n) = 2T(n/2) + c\,n\ (2/2)$

Passo 4. Sommando i costi di tutti i livelli:

$$T(n) = \sum_{i=0}^{\log_2 n - 1} c\,n + n\,T(1)$$

$$= c\,n \log_2 n + n\,T(1)$$

$$= \Theta(n \log_2 n)$$

- Per $c = 1$ e $T(1) = 1$ otteniamo di nuovo $T(n) = n \log_2 n + n$, come con il metodo di espansione

Costi costanti per livello: tutti i livelli costano uguale

- La soluzione di $T(n) = 2T(n/2) + c\,n$ è

$$T(n) = \Theta(n \log_2 n)$$

- Tutti i livelli hanno lo **stesso costo** $c\,n$
- Ci sono $\Theta(\log_2 n)$ livelli

Un albero sbilanciato: $T(n) = T(n/3) + T(2n/3) + c\,n$

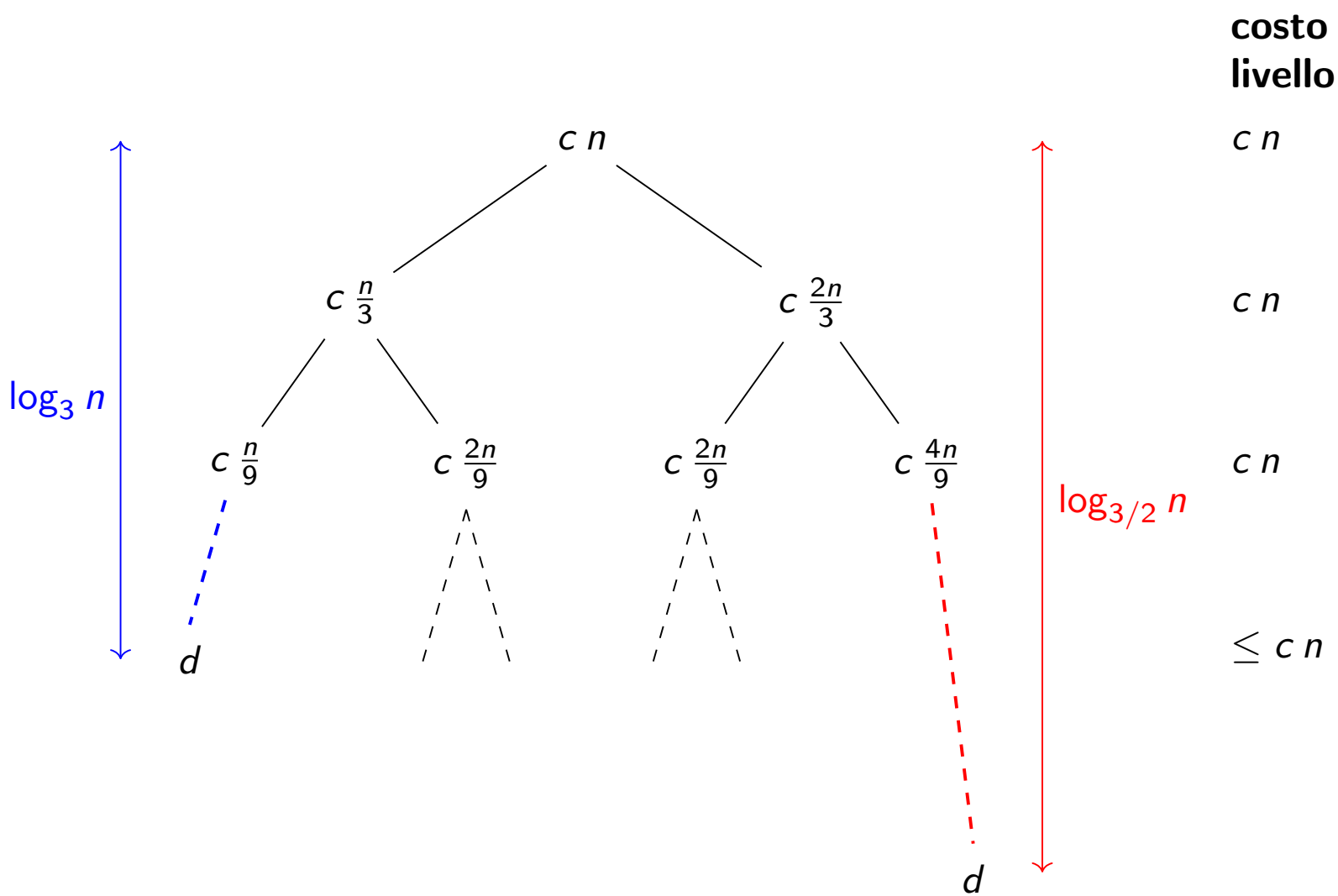
- Consideriamo la ricorrenza ($c, d > 0$ costanti), definita per ogni numero reale $n > 0$:

$$T(n) = \begin{cases} d & \text{se } 0 < n < 1 \\ T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + c\,n & \text{se } n \geq 1 \end{cases}$$

- Lavorare con dimensioni reali ci permette di evitare le parti intere inferiori e superiori

- Nell'albero di ricorsione il cammino più corto dalla radice a una foglia (blu) ha lunghezza circa $\log_3 n$, il più lungo (rosso) circa $\log_{3/2} n$

Albero di ricorsione di $T(n) = T(n/3) + T(2n/3) + c\,n$



La forma dell'albero sbilanciato (1/2)

- Ogni nodo interno di dimensione s ha due figli di dimensioni $\frac{s}{3}$ e $\frac{2s}{3}$, la cui somma è s

  Quindi le dimensioni dei nodi a una data profondità sommano al più a n (esattamente n se il livello è completo)

- Il **cammino più corto** dalla radice a una foglia è quello più a sinistra:

$$n \to \frac{n}{3} \to \frac{n}{9} \to \cdots$$

  Un nodo a profondità i su questo cammino ha dimensione $\frac{n}{3^i}$, che è ≥ 1 finché $i \leq \log_3 n$

La forma dell'albero sbilanciato (2/2)

- Il **cammino più lungo** è quello più a destra:

$$n \to \frac{2n}{3} \to \frac{4n}{9} \to \cdots$$

  Un nodo a profondità i su questo cammino ha dimensione $\left(\frac{2}{3}\right)^i n$, che è ≥ 1 finché $i \leq \log_{3/2} n$

- A differenza degli esempi precedenti, l'albero **non è completo**: le foglie si trovano a profondità diverse, comprese (approssimativamente) tra $\log_3 n$ e $\log_{3/2} n$

Esempio: limitazioni per l'albero sbilanciato (1/4)

Limitazione superiore: nodi interni.

- Ogni nodo a profondità i ha dimensione al più $\left(\frac{2}{3}\right)^i n$
- Un nodo interno ha dimensione ≥ 1, quindi i nodi interni compaiono solo a profondità $i \leq \log_{3/2} n$
- A ciascuna profondità, i nodi interni hanno dimensione totale al più n, quindi il loro costo totale è al più $c\,n$
- Pertanto il costo dei nodi interni è al più

$$c\,n\left(\log_{3/2} n + 1\right)$$

Esempio: limitazioni per l'albero sbilanciato (2/4)

Limitazione superiore: foglie.

- Una foglia ha dimensione < 1, ma il suo genitore ha dimensione ≥ 1, quindi una foglia ha dimensione $\geq \frac{1}{3}$
- Le dimensioni di tutte le foglie sommano esattamente a n (ogni nodo interno ripartisce la sua dimensione tra i suoi figli)
- Quindi ci sono al più $3n$ foglie, con costo totale al più $3\,d\,n = O(n)$

Pertanto

$$T(n) \leq c\,n\left(\log_{3/2} n + 1\right) + 3\,d\,n = O(n \log_2 n),$$

poiché $\log_{3/2} n = \frac{\log_2 n}{\log_2(3/2)}$ differisce da $\log_2 n$ per un fattore costante.

Esempio: limitazioni per l'albero sbilanciato (3/4)

Limitazione inferiore.

- Ogni nodo a profondità i ha dimensione almeno $\frac{n}{3^i}$
- Quindi, per ogni profondità $i \leq \log_3 n$, *tutti* i nodi a profondità i hanno dimensione ≥ 1: sono nodi interni, il livello è completo (2^i nodi) e le loro dimensioni sommano esattamente a n
- Il costo di ciascuno di questi livelli è esattamente $c\,n$
- Ci sono $\lfloor \log_3 n \rfloor + 1 \geq \log_3 n$ tali livelli (profondità $0, 1, \ldots, \lfloor \log_3 n \rfloor$), quindi

$$T(n) \geq c\,n \log_3 n = \Omega(n \log_2 n)$$

Esempio: limitazioni per l'albero sbilanciato (4/4)

Verifica numerica ($c = d = 1$): calcolando $T(n)$ esattamente dalla ricorrenza,

n	$n \log_3 n$	$T(n)$	$n(\log_{3/2} n + 1) + 3n$
100	419.2	≈ 929.2	1 535.8
1000	6 287.7	$\approx 13\,002.3$	21 036.6

- Entrambi i valori di $T(n)$ sono compresi tra la limitazione inferiore $n \log_3 n$ e la limitazione superiore $n(\log_{3/2} n + 1) + 3n$

L'albero sbilanciato: cammino più lungo e più corto

- La soluzione di $T(n) = T(n/3) + T(2n/3) + c\,n$ è

$$T(n) = \Theta(n \log_2 n)$$

- Ogni livello costa al più $c\,n$
- La limitazione superiore usa il **cammino più lungo** ($\log_{3/2} n$) e quella inferiore usa il **cammino più corto** ($\log_3 n$)
- Entrambi sono $\Theta(\log_2 n)$

Osservazione: l'albero è lontano dall'essere completo

- Se l'albero fosse un albero binario completo di altezza $\log_{3/2} n$, avrebbe

$$2^{\log_{3/2} n} = n^{\log_{3/2} 2} \approx n^{1.71}$$

  foglie, che è $\omega(n \log_2 n)$

- L'argomento precedente mostra che l'albero è lontano dall'essere completo: il numero effettivo di foglie è $O(n)$

- Quando le foglie non sono facili da contare, un approccio sicuro è usare l'albero di ricorsione solo per *ipotizzare* la soluzione e poi dimostrarla con il metodo di sostituzione

Esempio: risolvere $T(n) = 4T(n/2) + n$ (1/3)

Consideriamo la ricorrenza

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 4\,T\left(\frac{n}{2}\right) + n & \text{se } n > 1 \end{cases}$$

dove n è una potenza esatta di 2.

Passi 1 e 2.

- A profondità i ci sono 4^i nodi, ciascuno di dimensione $\frac{n}{2^i}$ e costo $\frac{n}{2^i}$
- Il costo del livello è

$$4^i \frac{n}{2^i} = 2^i\, n$$

Esempio: risolvere $T(n) = 4T(n/2) + n$ (2/3)

Passo 3. La dimensione raggiunge 1 a profondità $\log_2 n$, dove ci sono $4^{\log_2 n} = n^2$ foglie:

profondità	numero di nodi	costo di ciascun nodo	costo del livello
0	1	n	n
1	4	$n/2$	$2n$
2	16	$n/4$	$4n$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	4^i	$n/2^i$	$2^i n$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\log_2 n$	$4^{\log_2 n} = n^2$	$T(1) = 1$	n^2

Esempio: risolvere $T(n) = 4T(n/2) + n$ (3/3)

Passo 4. I costi *crescono* geometricamente scendendo lungo l'albero. Usando $\sum_{i=0}^{k-1} 2^i = 2^k - 1$,

$$T(n) = \sum_{i=0}^{\log_2 n - 1} 2^i\, n + n^2 = n\left(2^{\log_2 n} - 1\right) + n^2$$

$$= n(n-1) + n^2 = 2n^2 - n.$$

- Quindi $T(n) = 2n^2 - n = \Theta(n^2)$: infatti $\lim_{n\to\infty} \frac{2n^2-n}{n^2} = 2 \in \mathbb{R}_{>0}$
- Verifica: $T(2) = 4 + 2 = 6$, $T(4) = 24 + 4 = 28$, $T(8) = 112 + 8 = 120 = 2 \cdot 64 - 8$

Costi crescenti per livello: dominano le foglie

- La soluzione di $T(n) = 4T(n/2) + n$ è

$$T(n) = \Theta(n^2)$$

- I costi dei livelli crescono geometricamente
- Il costo totale è dominato dalle **foglie**, il cui numero è

$$4^{\log_2 n} = n^{\log_2 4} = n^2$$

Riepilogo (1/2)

Il metodo dell'albero di ricorsione:

- ogni nodo è il costo di un sottoproblema; si sommano i costi livello per livello
- dimensioni e nodi a profondità i, costo di un livello, profondità e foglie, somma
- risolve la ricorrenza o produce un'ipotesi per il metodo di sostituzione

Alberi bilanciati:

- $3T(n/4) + c\,n^2$: costi decrescenti per livello, domina la radice, $\Theta(n^2)$
- $2T(n/2) + c\,n$: costi costanti per livello, $\Theta(n \log_2 n)$
- $4T(n/2) + n$: costi crescenti per livello, dominano le foglie, $\Theta(n^2)$

Riepilogo (2/2)

Un albero sbilanciato:

- $T(n/3) + T(2n/3) + c\,n$: ogni livello costa al più $c\,n$
- limitazione superiore con il cammino più lungo $\log_{3/2} n$, inferiore con il cammino più corto $\log_3 n$
- soluzione $\Theta(n \log_2 n)$; il numero di foglie è $O(n)$

Un approccio sicuro:

- quando le foglie non sono facili da contare, si ipotizza con l'albero e si dimostra con la sostituzione

Verso il Master theorem

Introduzione: Verso il Master theorem

- I tre alberi bilanciati della sezione precedente sono istanze della stessa figura
- Disegniamo ora l'albero di ricorsione di una generica ricorrenza in forma standard
- I tre comportamenti che vi osserviamo sono il punto di partenza del Master theorem, più avanti nel corso

Una generica ricorrenza in forma standard

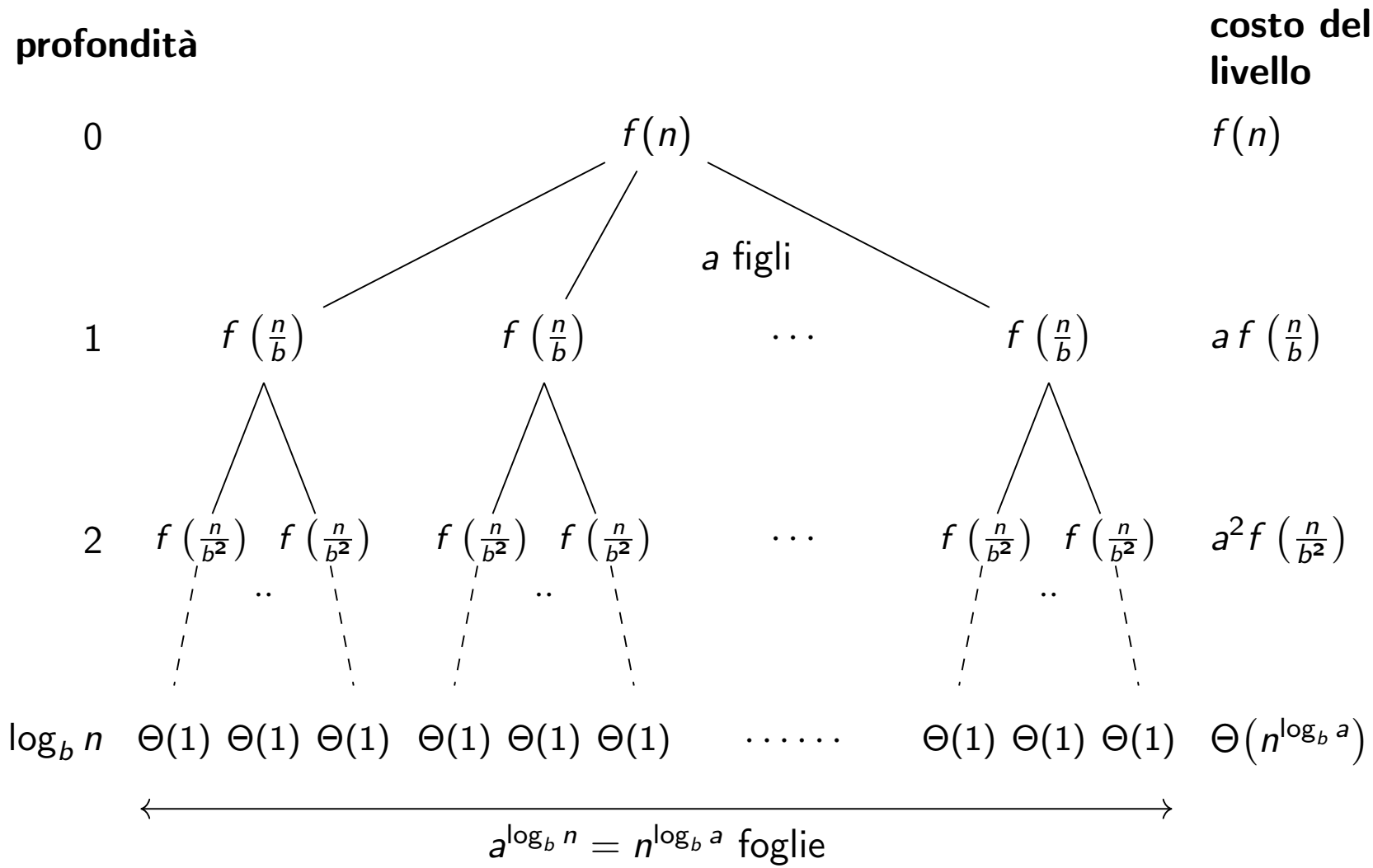
- Consideriamo

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ a\,T\left(\frac{n}{b}\right) + f(n) & \text{se } n > 1 \end{cases}$$

  con $a \geq 1$, $b > 1$ costanti e n potenza esatta di b

- Ne disegniamo l'albero di ricorsione, per n potenza esatta di b

Albero di ricorsione di $T(n) = a\,T(n/b) + f(n)$



Costi nell'albero di ricorsione generico (1/2)

- A profondità j ci sono a^j nodi, ciascuno corrispondente a un sottoproblema di dimensione $\frac{n}{b^j}$ con costo $f\left(\frac{n}{b^j}\right)$
- Il costo del livello è

$$a^j f\left(\frac{n}{b^j}\right)$$

- La dimensione del sottoproblema raggiunge 1 a profondità $\log_b n$
- Il numero di foglie è, usando $a^{\log_b n} = n^{\log_b a}$,

$$a^{\log_b n} = n^{\log_b a},$$

e il loro costo totale è $\Theta\left(n^{\log_b a}\right)$

Costi nell'albero di ricorsione generico (2/2)

- Sommando i costi di tutti i livelli:

$$T(n) = \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) \qquad (1)$$

- La soluzione dipende da come i costi dei livelli $a^j f(n/b^j)$ evolvono al variare di j

- Cioè dal confronto tra
  - $f(n)$, il costo della radice, e
  - $n^{\log_b a}$, il costo delle foglie

Tre comportamenti (1/2)

Ci sono tre possibili comportamenti di (1), tutti già incontrati:

1. **Dominano le foglie**: i costi dei livelli crescono geometricamente scendendo lungo l'albero, e $T(n) = \Theta\left(n^{\log_b a}\right)$
   Esempio: $T(n) = 4T(n/2) + n$, con $n^{\log_2 4} = n^2$ foglie, $T(n) = \Theta(n^2)$

2. **Tutti i livelli hanno (circa) lo stesso costo**: il costo di ciascuno dei $\log_b n + 1$ livelli è $\Theta\left(n^{\log_b a}\right)$, e $T(n) = \Theta\left(n^{\log_b a} \log_2 n\right)$
   Esempio: $T(n) = 2T(n/2) + c\,n$, con $n^{\log_2 2} = n$, $T(n) = \Theta(n \log_2 n)$

Tre comportamenti (2/2)

3. **Domina la radice**: i costi dei livelli decrescono geometricamente scendendo lungo l'albero, e $T(n) = \Theta(f(n))$
   Esempio: $T(n) = 3T(n/4) + c\,n^2$, con $f(n) = c\,n^2$, $T(n) = \Theta(n^2)$

In ciascun caso la soluzione è decisa dal confronto tra $f(n)$ e $n^{\log_b a}$.

Il Master theorem

- Il **Master theorem**, studiato più avanti nel corso, rende precisi questi tre comportamenti
- Confronta $f(n)$ con $n^{\log_b a}$
- Fornisce direttamente la soluzione di una ricorrenza in forma standard, senza disegnare l'albero di ricorsione

Riepilogo

L'albero di ricorsione generico:

- a^j nodi a profondità j, costo del livello $a^j f(n/b^j)$
- $n^{\log_b a}$ foglie, con costo totale $\Theta\left(n^{\log_b a}\right)$

Tre comportamenti:

- dominano le foglie: $\Theta\left(n^{\log_b a}\right)$
- tutti i livelli costano uguale: $\Theta\left(n^{\log_b a} \log_2 n\right)$
- domina la radice: $\Theta(f(n))$

Riepilogo: quale metodo quando

Introduzione: Riepilogo: quale metodo quando

- Abbiamo visto tre metodi per risolvere le ricorrenze, e il Master theorem sarà un quarto
- Ogni metodo si adatta a un tipo diverso di ricorrenza
- Raccogliamo qui quando usare ciascuno di essi

Quale metodo quando (1/2)

- **Metodo di espansione**: ricorrenze con un *solo* termine ricorsivo, per esempio $T(n-1)+f(n)$, $2T(n-1)+1$, $T(n/2)+c$, $2T(n/2)+n$

  Fornisce la forma chiusa esatta quando la sommatoria che ne risulta si può calcolare (serie aritmetica o geometrica)

- **Metodo di sostituzione**: quando abbiamo un'ipotesi e vogliamo una dimostrazione *rigorosa*, in particolare con parti intere e condizioni al contorno esplicite

  Si dimostrano O e Ω separatamente; i casi base si trattano scegliendo n_0; se l'induzione fallisce per un termine di ordine inferiore, si sottrae un termine di ordine inferiore dall'ipotesi. Si può combinare con un cambio di variabile

Quale metodo quando (2/2)

- **Metodo dell'albero di ricorsione**: per visualizzare i costi e ottenere un'ipotesi; ideale quando i termini ricorsivi hanno *dimensioni diverse* (alberi sbilanciati, come $T(n/3) + T(2n/3) + c\,n$)

  Si calcolano il costo per livello, la profondità, il numero di foglie, e si somma usando la serie geometrica

- **Master theorem** (più avanti nel corso): ricorrenze in *forma standard a* $T(n/b) + f(n)$

  Dà la risposta direttamente confrontando $f(n)$ con $n^{\log_b a}$. Non si applica a $T(n-1) + f(n)$ né alle suddivisioni sbilanciate

Riepilogo: punti chiave del capitolo (1/2)

- Una ricorrenza descrive una funzione $T(n)$ in termini dei suoi valori su input più piccoli; il numero di passi di un algoritmo ricorsivo ne soddisfa una
- Risolvere una ricorrenza significa trovarne la forma chiusa o l'ordine di grandezza; parti intere e condizioni al contorno si possono di solito trascurare
- Il metodo di espansione srotola la ricorrenza e somma: $T(n-1)+n=\Theta(n^2)$, $2T(n-1)+1=\Theta(2^n)$, $2T(n/2)+n=\Theta(n\log_2 n)$

Riepilogo: punti chiave del capitolo (2/2)

- Il metodo di sostituzione dimostra un'ipotesi per induzione; quando fallisce per un termine di ordine inferiore, si sottrae un termine di ordine inferiore dall'ipotesi
- Il metodo dell'albero di ricorsione somma i costi livello per livello; tratta anche alberi sbilanciati come $T(n/3) + T(2n/3) + c\,n = \Theta(n \log_2 n)$
- In forma standard, la soluzione dipende da chi domina, le foglie, tutti i livelli o la radice: il Master theorem lo rende preciso